Tsw

Programmer's guide

2026-10-27

Contents

index **3**
toggleswitch **3**
tsw **11**
License **17**

index

Keywords: index

The Toggle Switch Widget Package Tsw 1.5

by

Csaba Nemethi
csaba.nemethi@t-online.de

Contents

Tsw Programmer's Guide

The tsw::toggleswitch Command

toggleswitch

Keywords: toggleswitch

The `tsw::toggleswitch` Command

For Tsw Version 1.5

by

Csaba Nemethi
csaba.nemethi@t-online.de

Contents

- Quick Reference
- Detailed Reference
- Choosing between toggleswitch and (ttk::)checkbutton

Quick Reference

NAME

`tsw::toggleswitch` – Create and manipulate toggle switch widgets

SYNOPSIS

```
tsw::toggleswitch pathName ?options?
```

STANDARD OPTIONS

```
-class       -style         -underline
-cursor      -takefocus     -width
-font        -text
-foreground  -textvariable
```

WIDGET-SPECIFIC OPTIONS

```
-command command

-offvalue value

-onvalue value

-size 1|2|3

-variable variable
```

WIDGET COMMAND

```
pathName attrib ?name ?value name value ...??

pathName cget option

pathName configure ?option ?value option value ...??

pathName hasattrib name

pathName identify ?element? x y

pathName instate stateSpec ?script?

pathName state ?stateSpec?

pathName style

pathName switchstate ?boolean?

pathName toggle

pathName unsetattrib name
```

KEYWORDS

toggleswitch, trough, slider, widget

Detailed Reference

NAME

`tsw::toggleswitch` – Create and manipulate toggle switch widgets

SYNOPSIS

```
tsw::toggleswitch pathName ?options?
```

DESCRIPTION

The `tsw::toggleswitch` command creates a new window named `pathName` and makes it into a **toggleswitch** widget. Additional options, described below, may be specified on the command line or in the option database to configure aspects of the toggleswitch widget, such as its size and the Tcl script to execute whenever the switch state of the widget is toggled.

The `tsw::toggleswitch` command returns its `pathName` argument. At the time this command is invoked, there must not exist a window named `pathName`, but `pathName`'s parent must exist.

A toggleswitch is a mega-widget consisting basically of a horizontal **trough** and a **slider**, just like a ttk::scale widget. Actually, these elements belong to a ttk::scale contained in the widget. The trough is a fully rounded filled rectangle, and the slider is a filled circle or fully rounded filled rectangle contained in the trough. Both elements are rendered using scaling-aware SVG images. Their dimensions depend on the value of the `-size` configuration option, the current theme, the display's scaling level, and Tk's scaling factor given by `[tk scaling]`at the time the first toggleswitch widget is created. The widget also contains a ttk::label displaying an optional text to the scale's left; this child widget is only managed if the text specified with the `-text` or `-textvariable` option is a nonempty string.

Just like a light switch, a toggleswitch widget can have one of two possible **switch state**s: on or off. In the on state the slider is placed at the end of the trough, and in the off state at its beginning. As described in the DEFAULT BINDINGS section below, the user can toggle between these two states with the mouse or the `space` key.

The Tcl command associated with a toggleswitch widget has a very simple API. You can use the `switchstate` subcommand to change or query the widget's switch state. By using the `-command` configuration option, you can specify a script to execute whenever the widget's switch state gets toggled. Just like the (ttk::)checkbutton, toggleswitch widgets also support the `-offvalue`, `-onvalue`, and `-variable` options.

The colors used when drawing the trough and slider in the various widget states (such as `active`, `background`, `disabled`, `pressed`, and `selected`) depend on the current theme. The implementation creates these elements when needed with the aid of generic procedures for arbitrary themes and theme-specific ones for a few built-in themes. Applications can add *explicit* support for any theme `theme` by providing an appropriate command of the name `tsw::createElements_theme`.

STANDARD OPTIONS

```
-class       -style         -underline
-cursor      -takefocus     -width
-font        -text
-foreground  -textvariable
```

See the **ttk_widget** manual entry for details on the standard options. The default value of the `-class` option is `Toggleswitch`. The value of the `-style` option defaults to `Toggleswitch2`, corresponding to the `-size` option's default value `2`. Depending on the value of the `-size` option, the standard `-style` value is one of `Toggleswitch1`, `Toggleswitch2`, or `Toggleswitch3` (see below). The default value of the `-takefocus` option is `"ttk::takefocus"` (just like for most Tk themed widgets).

WIDGET-SPECIFIC OPTIONS

Command-Line Name:	-command
Database Name:	command
Database Class:	Command

Specifies a Tcl script to be evaluated at global scope whenever the switch state of the widget is toggled (programmatically, by invoking the `switchstate` or `toggle` subcommand, or interactively). The default is an empty string.

Command-Line Name:	-offvalue
Database Name:	offValue
Database Class:	OffValue

The value to store in the associated variable when the widget's switch state is set to off. Defaults to `0`.

Command-Line Name:	-onvalue
Database Name:	onValue
Database Class:	OnValue

The value to store in the associated variable when the widget's switch state is set to on. Defaults to `1`.

Command-Line Name:	-size
Database Name:	size
Database Class:	Size

Specifies the size identifier of the toggleswitch widget. The supported values are the strings `1`, `2` (the default), and `3`. If the current theme is `aqua` then these size IDs correspond to the control sizes "mini", "small", and "large" of the native toggle switches on macOS. For macOS versions 15 (Sequoia) and earlier the value `1` stands for the trough size of 26 x 15 pixels, the value `2` for the trough size of 32 x 18 pixels, and the value `3` identifies the trough size of 38 x 22 pixels. Starting with macOS 26 (Tahoe), the trough sizes are 29 x 13, 36 x 16, and 44 x 20 pixels, respectively. For all the other themes, on an unscaled screen the value `1` stands for the trough size of 32 x 16 pixels, the value `2` for the trough size of 40 x 20 pixels, and the value `3` identifies the trough size of 48 x 24 pixels, except that on Windows 10 and earlier, for compatibility with the native toggle switch, for the themes `vista`, `winnative`, and `xpnative` the unscaled trough width is 35, 44, and 53 pixels, respectively.

Note that by setting this option to a value `size`, the `-style` option's value will automatically change to `Toggleswitchsize` if its previous or requested value was one of `Toggleswitch1`,

Toggleswitch2, or Toggleswitch3, and to prefix.Toggleswitchsize if its previous or requested value was one of prefix.Toggleswitch1, prefix.Toggleswitch2, or prefix.Toggleswitch3.

Conversely, by setting the -style option to a value of the form Toggleswitchsize or prefix.Toggleswitchsize (where size is one of 1, 2, or 3), the -size option will automatically be set to size.

When configuring both options -size and -style, the former will take precedence over the latter, regardless of the order in which they were specified. For example, if you create the widget with

```
tsw::toggleswitch pathName -size 3 -style My.Toggleswitch3
```

or invoke

```
pathName configure -style My.Toggleswitch3
```

then the -style option will be set to My.Toggleswitch3 (and the -size option will have the value 3). On the other hand, if you create the widget with

```
tsw::toggleswitch pathName -style My.Toggleswitch3
```

then the -style option will have the value My.Toggleswitch2 rather than My.Toggleswitch3, because the widget creation triggers the default -size 2setting, which takes precedence over -style My.Toggleswitch3.

Command-Line Name:	-variable
Database Name:	variable
Database Class:	Variable

The name of a global variable whose value is linked to the toggleswitch. The widget's switch state changes to on when this variable is set to the value specified by the -onvalue option and to off otherwise. Defaults to the widget's pathname if not specified.

Note that, just like in the case of the (ttk::)checkbutton, toggling the widget's switch state by changing the value of this variable will *not* cause the script specified by the -command option to get executed.

WIDGET COMMAND

The tsw::toggleswitch command creates a new Tcl command whose name is pathName . This command may be used to invoke various operations on the widget. It has the following general form:

```
pathName option ?arg arg ...?
```

option and the args determine the exact behavior of the command. The following commands are possible for toggleswitch widgets:

pathName **attrib**?name ?value name value ...??

Queries or modifies the attributes of the widget. If no name is specified, the command returns a list of pairs, each of which contains the name and the value of an attribute for

pathName. If name is specified with no value, then the command returns the value of the one named attribute, or an empty string if no corresponding value exists (you can use the hasattrib subcommand to distinguish this case from the one that the value of an *existing* attribute is an empty string). If one or more name-value pairs are specified, then the command sets the given widget attribute(s) to the given value(s); in this case the return value is an empty string. name may be an arbitrary string.

pathName **cget** option
: Returns the current value of the configuration option given by option, which may have any of the values accepted by the tsw::toggleswitch command.

pathName **configure**?option ?value option value ...??
: Queries or modifies the configuration options of the widget. If no option is specified, the command returns a list describing all of the available options for pathName (see Tk_ConfigureInfo for information on the format of this list). If option is specified with no value, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no option is specified). If one or more option-value pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the return value is an empty string. option may have any of the values accepted by the tsw::toggleswitch command.

pathName **hasattrib** name
: Returns 1 if the attribute name exists and 0 otherwise.

pathName **identify**?element? x y
: See the **ttk_widget** manual entry. The coordinates are expected to be relative to the toggleswitch, but they are converted to values relative to the ttk::scale child and the return value will be the name of the corresponding style element of this child or an empty string.

pathName **instate** stateSpec ?script?
: See the **ttk_widget** manual entry.

pathName **state**?stateSpec?
: See the **ttk_widget** manual entry.

pathName **style**
: Returns the style used by the underlying ttk::scale widget. For Tk themed widgets this subcommand was introduced in Tk 8.7a4, but the toggleswitch widget provides it for all supported Tk versions.

pathName **switchstate**?boolean?
: Modifies or inquires the widget's switch state. If the optional argument is present then it must be a boolean (a numeric value, where 0 is false and anything else is true, or a string such as true/yes/on or false/no/off). If the widget's disabled state flag is set then the command returns an empty string immediately after checking the argument. Otherwise, if the argument is true then the command sets the widget's switch state to on by setting the underlying ttk::scale widget's selected flag, moving the slider to the end of the trough, and setting the associated variable to the value specified by the -onvalue option; if the argument is false then the command sets the widget's switch state to off by clearing the selected flag, moving the slider to the beginning of the trough, and setting the associated variable to the value specified by the -offvalue option.

  If the argument's value causes the widget's switch state to get toggled and the script

specified as the value of the `-command` option is a nonempty string then the command evaluates that script at global scope and returns its result; otherwise the return value is an empty string.

If the optional argument is not present then the command returns the widget's current switch state as `0` (off) or `1` (on). When a toggleswitch widget is created, its switch state is initialized with `0`.

`pathName` **toggle**
This convenience subcommand toggles the widget's switch state. It is logically equivalent to invoking the `switchstate` command with the argument `0` if the current switch state is on and with the argument `1` otherwise.

`pathName` **unsetattrib** `name`
Unsets the attribute `name`. Returns an empty string.

DEFAULT BINDINGS

The Tsw package replaces the default bindings of the ttk::scale contained in a toggleswitch widget with its own bindings as follows:

If the current theme is `aqua`:

1. By pressing mouse button 1 over the slider and then dragging the mouse with button 1 down until the pointer enters the trough, the slider moves smoothly to the opposite edge of the trough and the widget's switch state gets toggled. The same happens if mouse button 1 is pressed outside the slider and then the pointer leaves the widget horizontally with button 1 down.
2. By pressing mouse button 1 anywhere within the widget and then releasing it over the widget without previously moving the slider, the latter moves smoothly to the opposite edge of the trough and the widget's switch state gets toggled.
3. When the widget has the input focus, the `space` key causes its switch state to get toggled.

If the current theme is different from `aqua`:

1. By pressing mouse button 1 anywhere within the widget and then dragging the mouse with button 1 down, the slider moves in the same (horizontal) direction as the pointer. By releasing the button, the switch state is set to off or on, depending on the slider's position relative to the middle of the widget.
2. By pressing mouse button 1 anywhere within the widget and then releasing it over the widget without previously dragging the mouse horizontally, the widget's switch state gets toggled.
3. When the widget has the input focus, the `space` key causes its switch state to get toggled.

If the widget's `disabled` state flag is set then none of the above actions occur.

WIDGET STATES

The widget sets the `selected` state whenever the switch state changes to on and clears it otherwise. The default bindings set and clear the `active` and `pressed` state flags.

STYLING OPTIONS

The default class name for a `tsw::toggleswitch` is `Toggleswitch`.

Dynamic states: `active`, `background`, `disabled`, `pressed`, `selected`.

Styling options configurable with `ttk::style` for `Toggleswitch1`, `Toggleswitch2`, and `Toggleswitch3`:

-padding

padding

Defaults to

0.75p

.

The ttk::scale and the ttk::label displaying an optional text are children of a ttk:frame having the layout `ToggleswitchHull`, which contains a `focus` element for all themes, except `aqua`. Styling options configurable with `ttk::style` for `ToggleswitchHull`:

-focuscolor

color

The default is theme-specific.

-focussolid

boolean

Defaults to true for the

classic

theme and false for all the others. This option was introduced in Tk 8.6.15; in earlier Tk versions it has no effect.

-focusthickness

amount

The default is

1

. The value may have any of the forms acceptable to

Tk_GetPixels

.

For the `aqua` theme these focus-related options are simply ignored.

See the **ttk_style** manual page for information on how to configure ttk styles. KEYWORDS toggleswitch, trough, slider, widget

---

Choosing between toggleswitch and (ttk::)checkbutton

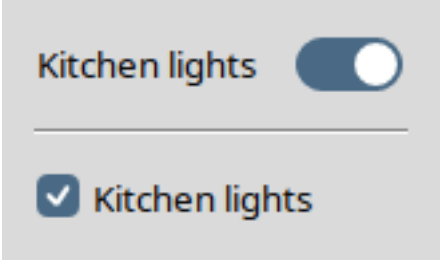
This section is based on the Microsoft Learn article *Toggle switches*.

For some actions, either a toggleswitch or a checkbutton might work. To decide which control would work better, follow these tips:

- Use a toggleswitch for binary settings that work like actions by taking effect immediately after the user flips the switch.

  In the following example, for turning the kitchen lights on, you should use a toggleswitch rather

than a checkbutton.



compare

- For optional ("nice to have") items use checkbuttons.
- Use checkbuttons when the user can select multiple items that are related to a single setting or feature.
- Use a checkbutton when the user has to perform extra steps for changes to be effective. For example, if the user must click a "Submit" or "Next" button to apply changes, use a checkbutton.

tsw

Keywords: tsw

Tsw Programmer's Guide

For Tsw Version 1.5

by

Csaba Nemethi
csaba.nemethi@t-online.de

Contents

Overview
- What Is Tsw?
- How to Get It?
- How to Install It?
- How to Use It?

Examples
- Tsw Demo
- Tablelist Editing Options

Overview

What Is Tsw?

Tsw stands for **T**oggle **sw**itch and is a library package for Tcl/Tk versions 8.6 or higher. If the version is 8.6 then in addition it is required that the tksvg extension can be loaded into the interpreter (Tk versions 8.7 and 9.0 or higher have built-in SVG support). The package is written in pure Tcl/Tk code and contains:

- the implementation of the toggle switch mega-widget toggleswitch, including a general utility module for mega-widgets;
- two richly commented demo scripts containing the typical steps needed to create and handle toggleswitch widgets;
- this tutorial;
- a reference page in HTML format.

A toggleswitch is a mega-widget consisting basically of a horizontal trough (a fully rounded filled rectangle) and a slider (a filled circle or fully rounded filled rectangle contained in the trunk). The widget also contains a ttk::label displaying an optional text to the trough's left; this child widget is only managed if the text specified with the `-text` or `-textvariable` option is a nonempty string.

A toggleswitch widget can have one of two possible switch states: on or off. In the on state the slider is placed at the end of the trough, and i https://core.tcl-lang.org/tklibhttps://core.tcl-lang.org/tklibn the off state at its beginning. The user can toggle between these two states with the mouse or the space key.

You can use the `switchstate` subcommand of the Tcl command associated with a toggleswitch to change or query the widget's switch state. By using the `-command` configuration option, you can specify a script to execute whenever the switch state of the widget gets toggled. For compatibility with the (ttk::)checkbutton, toggleswitch widgets also support the `-offvalue`, `-onvalue`, and `-variable` options.

How to Get It?

Tsw is available for free download from the Web page

https://www.nemethi.de

The distribution file is `tsw1.5.tar.gz` for UNIX and `tsw1_5.zip` for Windows. These files contain the same information, except for the additional carriage return character preceding the linefeed at the end of each line in the text files for Windows.

Tsw is also included in tklib, which has the address

https://core.tcl-lang.org/tklib

How to Install It?

Install the package as a subdirectory of one of the directories given by the `auto_path` variable. For example, you can install it as a subdirectory of the `lib` directory within your Tcl/Tk installation (at the same level as the `tk8.X` or `tk9.X` subdirectory).

To install Tsw *on UNIX*, `cd` to the desired directory and unpack the distribution file `tsw1.5.tar.gz`:

```
gunzip -c tsw1.5.tar.gz | tar -xf -
```

On most UNIX systems this can be replaced with

```
tar -zxf tsw1.5.tar.gz
```

Both commands will create a directory named `tsw1.5`, with the subdirectories `demos`, `doc`, and `scripts`.

On Windows, use WinZip or some other program capable of unpacking the distribution file

`tsw1_5.zip` into the directory `tsw1.5`, with the subdirectories `demos`, `doc`, and `scripts`.

Notice that in tklib the Tsw `demos` directory is replaced with the subdirectory `tsw` of the `examples` directory. Please take this into account when reading the examples below.

How to Use It?

To be able to access the commands and variables of the Tsw package, your scripts must contain one of the lines

```
package require tsw ?version?
package require Tsw ?version?
```

You can use either one of the two statements above because the file `tsw.tcl` contains both lines

```
package provide tsw ...
package provide Tsw ...
```

You are free to remove one of these two lines from `tsw.tcl` if you want to prevent the package from making itself known under two different names. Of course, by doing so you restrict the argument of `package require`to a single name.

Since the Tsw package is implemented in its own namespace called `tsw`, you must either invoke the

```
namespace import tsw::toggleswitch
```

command to import the only public *procedure* of the `tsw` namespace, or use the qualified name `tsw::toggleswitch`. In the examples below we have chosen the latter approach.

To access Tsw *variables*, you *must* use qualified names. There are only two Tsw variables that are designed to be accessed outside the `tsw` namespace:

- The variable `tsw::version` holds the current version number of the Tsw package.
- The variable `tsw::library` holds the location of the Tsw installation directory.

Examples

Tsw Demo

The script `TswDemo.tcl` in the `demos` directory creates four toggleswitch widgets and shows how their appearance depends on the current theme, which can be selected with the aid of the menu associated with a ttk::menubutton widget. In addition, it demonstrates how to specify a script to execute whenever the switch state of a toggleswitch gets toggled.



Here is the code that creates the four toggleswitch widgets:

```
package require Tk
package require tsw

wm title . "Tsw Demo"

ttk::frame .tf
ttk::frame .bf

#
# Create 3 toggleswitch widgets having different values of the -size option
#
set l1 [ttk::label .tf.l1 -text "Toggle switch of size 1"]
set sw1 [tsw::toggleswitch .tf.sw1 -size 1]
set l2 [ttk::label .tf.l2 -text "Toggle switch of size 2"]
set sw2 [tsw::toggleswitch .tf.sw2 -size 2]
$sw2 switchstate 1
set l3 [ttk::label .tf.l3 -text "Toggle switch of size 3"]
set sw3 [tsw::toggleswitch .tf.sw3 -size 3]

#
# Create a toggleswitch widget of default size and set its -command option
#
set l4 [ttk::label .tf.l4 -text "Enable/disable above widgets"]
set sw4 [tsw::toggleswitch .tf.sw4]
$sw4 switchstate 1
$sw4 configure -command [list toggleWidgetsState $sw4]
```

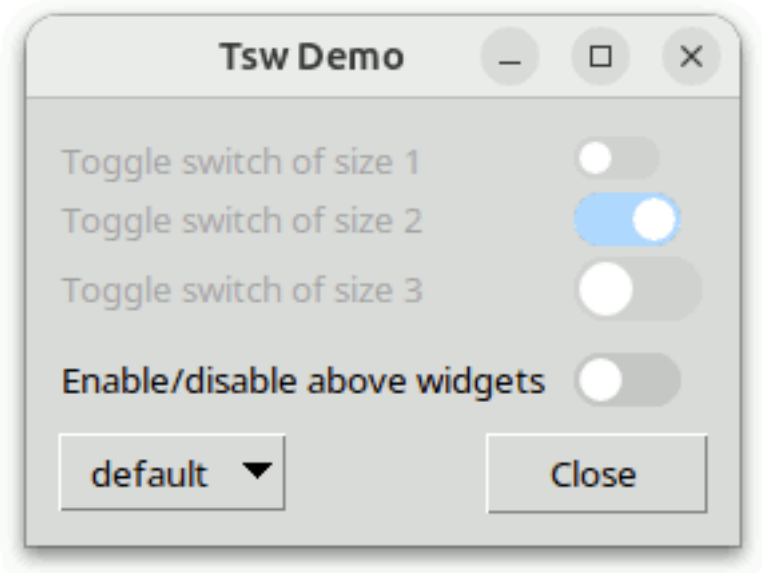
We create the toggleswitch widgets by invoking the `tsw::toggleswitch` command. For the first three toggleswitch widgets we also set the `-size` option to `1`, `2`, and `3`, respectively. For the last toggleswitch we don't explicitly set this option, hence it will have its default value `2`. As seen in the screenshots, in the case of the `aqua` theme the colors used when drawing the toggleswitch widgets also depend on the system appearance (light mode or dark mode) and the accent color. Note that the first two screenshots were taken on macOS 15 (Sequoia), while the third one on macOS 26 (Tahoe).

For two of the four toggleswitch widgets we change the switch state from the default value `0` (off) to `1` (on) by invoking the `switchstate` subcommand of the associated Tcl command. In addition, for the last toggleswitch we set the `-command` option to a script that will be executed whenever the widget's switch state gets toggled. This script invokes the procedure `toggleWidgetsState` implemented as follows:

```
#------------------------------------------------------------------------------
# toggleWidgetsState
#
# Enables/disables the widgets in the first 3 grid rows, depending on the
# switch state of the specified toggleswitch widget.
#------------------------------------------------------------------------------
proc toggleWidgetsState sw {
    global l1 l2 l3 sw1 sw2 sw3
    set stateSpec [expr {[$sw switchstate] ? "!disabled" : "disabled"}]
    foreach w [list $l1 $l2 $l3 $sw1 $sw2 $sw3] {
        $w state $stateSpec
    }
}
```

This time the `switchstate` subcommand is invoked without the optional argument, hence it returns the toggleswitch widget's current switch state.

For the `default` theme, after changing the switch state of the last toggleswitch from on to off, the window will look as shown in the screenshot below:



Tsw Demo

The rest of the code is not Tsw-specific and for this reason is not shown here.

Tablelist Editing Options

The script `EditingOpts.tcl` in the `demos` directory is a slightly adapted version of the Tablelist demo script `tileWidgets2.tcl`, which demonstrates the interactive tablelist cell editing with the aid of various Ttk widgets and the toggleswitch widget. The additional functionality in this version is implemented in the procedures `configEditing` and `applySwitchState`. The first one, triggered by the "Configure Editing" button, opens a toplevel window containing toggleswitch widgets for configuring the editing-related tablelist options having boolean values, proposed over the years by Tablelist users. This is a comfortable way to test the effect of setting/clearing these boolean options.





The `configEditing` procedure is shown below:

```
package require Tk
package require tsw
package require tablelist_tile

. . .

#------------------------------------------------------------------------------
# configEditing
#
# Configures the editing-related tablelist options having boolean values with
# the aid of toggleswitch widgets.
#------------------------------------------------------------------------------
proc configEditing tbl {
    set top .top
    if {[winfo exists $top]} {
        raise $top
        focus $top
        return ""
    }

    toplevel $top
    wm title $top "Editing Options"

    set tf [ttk::frame $top.tf]
    set bf [ttk::frame $top.bf]

    #
    # Create the widgets corresponding to the
    # editing-related options with boolean values
    #
    set row 0
    foreach opt {
        -autofinishediting
        -editendonfocusout
        -editendonmodclick
        -editselectedonly
        -forceeditendcommand
        -instanttoggle
        -showeditcursor
    } {
        lassign [$tbl configure $opt] option dbName dbClass default current
        set defaultStr [expr {$default ? "on" : "off"}]

        set sw [tsw::toggleswitch $tf.sw$row -text "$opt ($defaultStr)"]
        if {$current != $default} {
            $sw configure -foreground red2
        }

        $sw switchstate $current        ;# sets the switch state to $current
```

```
        $sw attrib default $default     ;# saves $default as attribute value
        $sw configure -command [list applySwitchState $sw $tbl $opt]
        pack $sw -padx 9p -pady {0 3p} -fill x

        incr row
    }

    . . .
}
```

For each of the 7 editing-related options with boolean values, the procedure displays the option's name and default value in a toggleswitch widget and sets the latter's switch state to the option's current value. In addition, it invokes the `attrib` subcommand of the Tcl command associated with the toggleswitch to save the default as the value of the widget's attribute of name `default`. In this way, the widget "remembers" the default value in an object-oriented manner and can retrieve it later without needing any external resources. For increased user-friendliness, the text of the toggleswitch is displayed in the `red2` foreground color if the option's current value is different from the default one.

Whenever the toggleswitch widget's switch state gets toggled, the script specified as the value of its `-command` option invokes the `applySwitchState` procedure shown below:

```
#-------------------------------------------------------------------------
# applySwitchState
#
# Sets the configuration option opt of the tablelist tbl and the foreground
# color of the toggleswitch widget sw according to its switch state.
#-------------------------------------------------------------------------
proc applySwitchState {sw tbl opt} {
    set switchState [$sw switchstate]
    $tbl configure $opt $switchState

    set fgColor [expr {$switchState == [$sw attrib default] ? "" : "red2"}]
    $sw configure -foreground $fgColor
}
```

We set the specified tablelist option to the toggleswitch widget's switch state, and also the widget's foreground color, depending on the switch state and the option's default value, which we retrieve by using the `attrib` toggleswitch subcommand.

Notice that in this example we specify the texts of the "Editing Options" window as the values the `-text` option of the toggleswitch widgets, unlike in the previous example, in which we used extra ttk::label widgets displayed to the left of the corresponding toggleswitches.

License

These pages were converted from the original manual pages to Markdown (TIP 700 format). The text is unchanged; only markup without a place in the format (links without a target, layout images) may be lost. For parts under the Mozilla Public License 1.1 (tDOM) this is the description of modifications required by section 3.3.

This collection is not an official publication of the projects. Each part is under the terms of its original, which follow. The copyright notices of the original pages stay with each page.

Toggle switch widget package Tsw 1.5 Copyright (c) 2025-2026 Csaba Nemethi (E-mail: csaba.nemethi@t-online.de)

This library is free software; you can use, modify, and redistribute it for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions.

This software is distributed WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Index

A

attrib 7

C

cget 8
Choosing between toggleswitch and (ttk::)checkbutton 10
configure 8

D

DEFAULT BINDINGS 9

H

hasattrib 8

I

identify 8
index 3
instate 8

S

state 8
style 8
STYLING OPTIONS 9
switchstate 8

T

Tablelist Editing Options 15
toggle 9
toggleswitch 3
tsw 11
Tsw Demo 13

U

unsetattrib 9

W

WIDGET STATES 9